

Assignment 3: Naive Bayes

Objective

- Train a naive bayes model on a sentiment analysis task
- Test using your model
- Compute ratios of positive words to negative words
- Do some error analysis
- Predict on your own tweet

Dataset

Use the provided **bayes_features.csv** file.

The dataset contains the following columns:

Column	Description
positive	Positive Naïve Bayes feature
negative	Negative Naïve Bayes feature
sentiment	Sentiment label: 1 = Positive, 0 = Negative

Dataset Setup

If you are using Google Colab, run the following cell to install/load the required NLTK datasets:

```
import nltk
```

```
nltk.download('stopwords')  
nltk.download('twitter_samples')
```

Part 1: Load and Explore the Dataset

Load the **bayes_features.csv** file into a Pandas DataFrame.

Tasks

1. Display the first five rows of the dataset.
2. Display the shape of the dataset.
3. Display the column names.
4. Display basic statistical information about the dataset.
5. Count the number of positive and negative tweets.

Questions

Q1. How many tweets are present in the dataset?

Q2. How many positive tweets are present?

Q3. How many negative tweets are present?

Q4. What are the minimum and maximum values of the **positive** feature?

Q5. What are the minimum and maximum values of the **negative** feature?

Part 2: Process the Data

For any machine learning project, once you've gathered the data, the first step is to process it to make useful inputs to your model.

- **Remove noise:** You will first want to remove noise from your data -- that is, remove words that don't tell you much about the content. These include all common words like 'I, you, are, is, etc...' that would not give us enough information on the sentiment.
- We'll also remove stock market tickers, retweet symbols, hyperlinks, and hashtags because they can not tell you a lot of information on the sentiment.
- You also want to remove all the punctuation from a tweet. The reason for doing this is because we want to treat words with or without the punctuation as the same word, instead of treating "happy", "happy?", "happy!", "happy," and "happy." as different words.
- Finally you want to use stemming to only keep track of one variation of each word. In other words, we'll treat "motivation", "motivated", and "motivate" similarly by grouping them within the same stem of "motiv-".

Part 2.1 Implementing your helper functions

To help train your naive bayes model, you will need to build a dictionary where the keys are a (word, label) tuple and the values are the corresponding frequency. Note that the labels we'll use here are 1 for positive and 0 for negative.

You will also implement a `lookup()` helper function that takes in the `freqs` dictionary, a word, and a label (1 or 0) and returns the number of times that word and label tuple appears in the collection of tweets.

For example: given a list of tweets `["i am rather excited", "you are rather happy"]` and the label 1, the function will return a dictionary that contains the following key-value pairs:

```
{ ("rather", 1): 2 ("happi", 1) : 1 ("excit", 1) : 1 }
```

- Notice how for each word in the given string, the same label 1 is assigned to each word.
- Notice how the words "i" and "am" are not saved, since it was removed by `process_tweet` because it is a stopword.
- Notice how the word "rather" appears twice in the list of tweets, and so its count value is 2.

Instructions

Create a function `count_tweets()` that takes a list of tweets as input, cleans all of them, and returns a dictionary.

- The key in the dictionary is a tuple containing the stemmed word and its class label, e.g. ("happi",1).
- The value the number of times this word appears in the given collection of tweets (an integer).

Hints

- Please use the `'process_tweet'` function that was imported above, and then store the words in their respective dictionaries and sets.
 - You may find it useful to use the `'zip'` function to match each element in `'tweets'` with each element in `'ys'`.
 - Remember to check if the key in the dictionary exists before adding that key to the dictionary, or incrementing its value.
 - Assume that the `'result'` dictionary that is input will contain clean key-value pairs (you can assume that the values will be integers that can be incremented). It is good practice to check the datatype before incrementing the value, but it's not required here.
-

Part 3: Train your model using Naive Baye

Part 4: Test your naive bayes

Now that we have the `logprior` and `loglikelihood`, we can test the naive bayes function by making predicting on some tweets!

Implement `naive_bayes_predict`

Instructions: Implement the `naive_bayes_predict` function to make predictions on tweets.

- The function takes in the `tweet`, `logprior`, `loglikelihood`.
 - It returns the probability that the tweet belongs to the positive or negative class.
 - For each tweet, sum up loglikelihoods of each word in the tweet.
 - Also add the logprior to this sum to get the predicted sentiment of that tweet.
-

Part 5: Filter words by Ratio of positive to negative counts

- Some words have more positive counts than others, and can be considered "more positive". Likewise, some words can be considered more negative than others.
- One way for us to define the level of positiveness or negativeness, without calculating the log likelihood, is to compare the positive to negative frequency of the word.
 - Note that we can also use the log likelihood calculations to compare relative positivity or negativity of words.
- We can calculate the ratio of positive to negative frequencies of a word.
- Once we're able to calculate these ratios, we can also filter a subset of words that have a minimum ratio of positivity / negativity or higher.
- Similarly, we can also filter a subset of words that have a maximum ratio of positivity / negativity or lower (words that are at least as negative, or even more negative than a given threshold).

Part 6: Error Analysis

In this part you will see some tweets that your model missclassified. Why do you think the misclassifications happened? Were there any assumptions made by the naive bayes model?

Part 6: Predict with your own tweet

In this part you can predict the sentiment of your own tweet.

Part 7: Visualizing Naive Bayes

Calculate the likelihoods for each tweet

For each tweet, we have calculated the likelihood of the tweet to be positive and the likelihood to be negative. We have calculated in different columns the numerator and denominator of the likelihood ratio introduced previously.

We did not include the code because this is part of this week's assignment. The '**bayes_features.csv**' file contains the final result of this process.

The cell below loads the table in a dataframe. Dataframes are data structures that simplify the manipulation of data, allowing filtering, slicing, joining, and summarization.

Part 8: Using Confidence Ellipses to interpret Naïve Bayes

In this section, we will use the [confidence ellipse](#) to give us an idea of what the Naïve Bayes model see.

A confidence ellipse is a way to visualize a 2D random variable. It is a better way than plotting the points over a cartesian plane because, with big datasets, the points can overlap badly and hide the real distribution of the data. Confidence ellipses summarize the information of the dataset with only four parameters:

- Center: It is the numerical mean of the attributes
- Height and width: Related with the variance of each attribute. The user must specify the desired amount of standard deviations used to plot the ellipse.
- Angle: Related with the covariance among attributes.

The parameter **n_std** stands for the number of standard deviations bounded by the ellipse. Remember that for normal random distributions:

- About 68% of the area under the curve falls within 1 standard deviation around the mean.
- About 95% of the area under the curve falls within 2 standard deviations around the mean.
- About 99.7% of the area under the curve falls within 3 standard deviations around the mean.

Submission Requirements

Submit **one Google Colab/Jupyter Notebook** containing:

1. Dataset loading and inspection
2. Scatter plot of the original data
3. Statistical analysis
4. Confidence ellipse visualization
5. Modified distribution
6. Confidence ellipse visualization of the modified distribution
7. Comparison table
8. Answers to **Q1–Q34**
9. Final analysis